JPEG Compression of a 24 bpp Image

Sahithyan K.
230557T

October 19, 2026

1 Introduction

JPEG is a lossy compression method for continuous-tone still images such as photographs. This report describes how the baseline JPEG encoder compresses a 24 bpp image, with a worked example and a Python implementation.

1.1 Terms

- bpp
  Bits per pixel. A 24 bpp image stores 3 channels (red, green, blue) of 8 bits each per pixel.
- Lossless compression
  The decoder recovers the original data exactly.
- Lossy compression
  The decoder recovers an approximation of the original. Information the viewer is unlikely to notice is discarded to reduce size.
- Compression ratio
  Original size divided by compressed size.
- PSNR
  Peak signal-to-noise ratio. A measure, in decibels (dB), of how close a decoded image is to the original. Higher means closer. See Section 2.10.
- JPEG
  Joint Photographic Experts Group. The name refers to the committee and to the standard it wrote, published as ITU-T T.81 [2].
- Baseline JPEG
  The standard's basic mode, where the image is encoded in a single pass with Huffman coding and 8 bits per channel. The standard also defines progressive, hierarchical and lossless modes. This report covers only baseline.
- JFIF
  JPEG File Interchange Format [3]. A minimal file format that wraps JPEG data.
- DCT
  Discrete cosine transform. A transform from pixel values to spatial frequency coefficients [4].
- Huffman coding
  A variable-length code that gives shorter bit strings to more frequent symbols [7].

1.2 History

- 1952
  Huffman publishes his minimum-redundancy code [7].
- 1974
  Ahmed, Natarajan and Rao introduce the DCT [4].
- 1986
  ISO and CCITT (now ITU-T) form the Joint Photographic Experts Group to define a standard for still image compression [1].
- 1992
  The standard is published as ITU-T T.81 [2]. ISO adopts it as ISO/IEC 10918-1 in 1994.
- 1992
  Eric Hamilton of C-Cube Microsystems publishes JFIF 1.02 [3].

The T.81 standard specifies how image data is compressed. It does not fix the color space, the pixel aspect ratio or the file layout. JFIF fills this gap by requiring YCbCr for color images and defining a header with the pixel density. Files with the `.jpg` or `.jpeg` extension are normally JFIF files.

1.3 Alternatives to JFIF

JFIF is not the only way to store JPEG data. The alternatives wrap the same compressed data with different metadata.

- Exif
  Exchangeable image file format [5]. Used by digital cameras and phones. Stores camera settings, timestamps and GPS location. Most photos from cameras are Exif files, not JFIF files.
- SPIFF
  Still Picture Interchange File Format, defined in ITU-T T.84 [6]. Intended to replace JFIF, with support for more color spaces and bit depths. Rarely used.

The compressed image data is the same in all 3 formats. Most software reads JFIF and Exif files alike.

1.4 Motivation

A 1920×1080 image at 24 bpp takes $1920 \times 1080 \times 3 = 6{,}220{,}800$ bytes, about 5.9 MiB, when stored raw. JPEG commonly reaches compression ratios of 10:1 or more with little visible loss [1].

2 Processing Steps

The baseline JPEG encoder [1] compresses a 24 bpp image (3 channels of 8 bits per pixel) in 7 steps, shown in Figure 1. Only chroma subsampling and quantization discard information by design.

RGB image → color conversion → chroma subsampling → 8×8 blocks + DCT
→ quantization → zigzag, RLE, Huffman coding → `.jpg`

Figure 1: Baseline JPEG encoder.

2.1 Color Space Conversion

YCbCr separates luma (brightness) from chroma (color). Human vision resolves brightness detail better than color detail, so the 2 can be compressed differently. R, G and B each mix both, so they cannot. JFIF [3] defines the conversion:

$$
\begin{aligned}
Y &= 0.299\,R + 0.587\,G + 0.114\,B \\
C_b &= 128 - 0.168736\,R - 0.331264\,G + 0.5\,B \\
C_r &= 128 + 0.5\,R - 0.418688\,G - 0.081312\,B
\end{aligned}
$$

Here:

- R, G, B: 8-bit red, green and blue values
- Y: luma
- C_b: blue-difference chroma
- C_r: red-difference chroma

2.2 Chroma Subsampling

Chroma subsampling stores C_b and C_r with fewer samples than Y. Human vision is less sensitive to color detail than to brightness detail, so the loss is rarely visible [9]. Y is never subsampled.

Every pixel always keeps its own Y sample. The schemes differ only in how many C_b and C_r samples are kept. The name $J{:}a{:}b$ describes a block 4 pixels wide and 2 pixels tall.

- J: pixels per row, always 4
- a: chroma samples in the first row
- b: new chroma samples in the second row

A value of $b = 0$ does not mean the second row has no color. The second row reuses the chroma samples of the first row.

Common schemes:

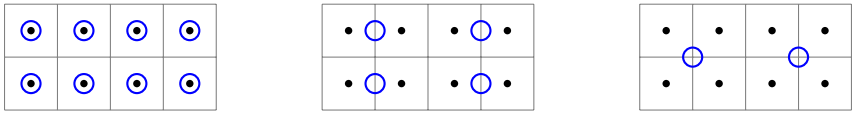
- 4:4:4
  $a = 4$, $b = 4$. Each of the 8 pixels has its own chroma sample. No subsampling.
- 4:2:2
  $a = 2$, $b = 2$. Each chroma sample covers 2 pixels side by side. Each row has 2 samples, giving 4 in total.
- 4:2:0
  $a = 2$, $b = 0$. Each chroma sample covers a 2×2 group of pixels. The 2 samples in the first row serve both rows, giving 2 in total. JPEG encoders use this by default.

Samples needed for the 4×2 block, counting Y, C_b and C_r:

Scheme	Y	C_b	C_r	Total
4:4:4	8	8	8	24
4:2:2	8	4	4	16
4:2:0	8	2	2	12

4:2:0 needs half the samples of 4:4:4.

Figure 2 shows where the samples sit in each scheme.



4:4:4 4:2:2 4:2:0

Figure 2: Sample positions in a 4×2 pixel block. Black dots are Y samples. Blue circles are shared C_b and C_r samples.

A 16×16 region reduces from $3 \times 256 = 768$ samples to $256 + 64 + 64 = 384$ in the same way. The data halves before any other compression step.

Example with 4:2:0, using a 4×4 region of C_b values. The region has 4 groups of 2×2 pixels.

$$\begin{bmatrix} 100 & 102 & 110 & 112 \\ 98 & 104 & 108 & 114 \\ 90 & 92 & 120 & 118 \\ 94 & 96 & 122 & 124 \end{bmatrix}$$

Averages:

- Top-left group
  $(100 + 102 + 98 + 104)/4 = 101$
- Top-right group
  $(110 + 112 + 108 + 114)/4 = 111$
- Bottom-left group
  $(90 + 92 + 94 + 96)/4 = 93$
- Bottom-right group
  $(120 + 118 + 122 + 124)/4 = 121$

Each group is replaced by its average, so 16 values become 4.

$$\begin{bmatrix} 100 & 102 & 110 & 112 \\ 98 & 104 & 108 & 114 \\ 90 & 92 & 120 & 118 \\ 94 & 96 & 122 & 124 \end{bmatrix} \longrightarrow \begin{bmatrix} 101 & 111 \\ 93 & 121 \end{bmatrix}$$

The decoder restores the original size by copying each value into its 2×2 group.

$$\begin{bmatrix} 101 & 101 & 111 & 111 \\ 101 & 101 & 111 & 111 \\ 93 & 93 & 121 & 121 \\ 93 & 93 & 121 & 121 \end{bmatrix}$$

The error (original minus restored) is:

$$\begin{bmatrix} -1 & 1 & -1 & 1 \\ -3 & 3 & -3 & 3 \\ -3 & -1 & -1 & -3 \\ 1 & 3 & 1 & 3 \end{bmatrix}$$

The largest error is 3 on a scale of 255, which is not visible to the human eye. The original values cannot be recovered.

The same happens to C_r. For these 4×4 pixels, the 48 samples (Y, C_b, C_r at 16 each) reduce to $16 + 4 + 4 = 24$.

2.3 Block Splitting and Level Shift

Each channel is split into 8×8 blocks. Edges are padded by repeating the last row or column when a dimension is not a multiple of 8. Each sample then has 128 subtracted, which moves the range to $[-128, 127]$ and centers it on 0. The result is called a **level-shifted sample**:

$$f(x, y) = p(x, y) - 2^{P-1}$$

where $p(x, y)$ is the pixel value and P is the bit depth ($P = 8$ here). The decoder adds 128 back at the end.

Every later step runs on each block independently. Heavy compression therefore leaves visible block edges, called blocking artifacts.

2.4 Discrete Cosine Transform

The 2D discrete cosine transform (DCT) [4] rewrites an 8×8 block as a weighted sum of 64 fixed cosine patterns of increasing spatial frequency.

$$F(u, v) = \tfrac{1}{4}\, C(u)C(v) \sum_{x=0}^{7} \sum_{y=0}^{7} f(x, y) \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

Here:

- $f(x, y)$: level-shifted sample at row x, column y
- $F(u, v)$: coefficient for the pattern of vertical frequency u and horizontal frequency v
- $C(0) = 1/\sqrt{2}$
- $C(k) = 1$ for $k > 0$

The coefficient $F(0, 0)$ is the **DC coefficient**. It is proportional to the block average. The other 63 coefficients are **AC coefficients**. The level shift is applied to zero-center the data. This transformation reduces the block average and the DC coefficient. AC coefficients are unchanged.

The DCT is invertible, apart from rounding error.

In natural images, neighboring pixels usually have similar values. A block therefore varies slowly. It is described mostly by smooth, low-frequency patterns. Most of its information ends up in a few coefficients near $F(0, 0)$. The 64 coefficients form an 8×8 array, indexed by u (row) and v (column). These low-frequency coefficients sit at the top-left of that array. The coefficients toward the bottom-right represent fast, fine detail. They are usually close to 0.

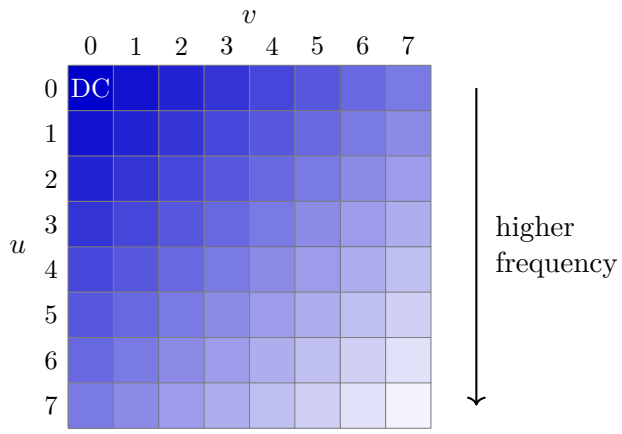


Figure 3: The 8×8 array of DCT coefficients $F(u, v)$. Darker cells hold larger coefficients in a typical block.

2.5 Quantization

Quantization divides each coefficient by the matching entry of a table Q and rounds to an integer.

$$F_q(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

Low frequencies get small divisors. High frequencies get large ones. Most high-frequency coefficients therefore round to 0. This rounding is the main irreversible loss in JPEG. The quality setting scales Q. A larger Q gives a smaller file and more visible damage.

The Y channel uses one table. The C_b and C_r channels use a second table with larger divisors. The eye is less sensitive to fine color detail than to fine brightness detail, so chroma can be rounded harder. The standard luminance table from Annex K of ITU-T T.81 [2] is:

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

2.6 Zigzag Scan and Run-Length Coding

The **zigzag scan** reads a quantized block diagonally from $(0, 0)$ to $(7, 7)$. Coefficients come out ordered from low to high frequency, which groups the zeros at the end. A single end-of-block (EOB) symbol replaces the trailing zeros.

DC and AC coefficients are coded differently.

- DC
  The difference from the previous block's DC is stored. Neighboring blocks have similar averages, so the differences are small.
- AC
  Each non-zero value is stored as a pair (r, s) followed by the value's bits.

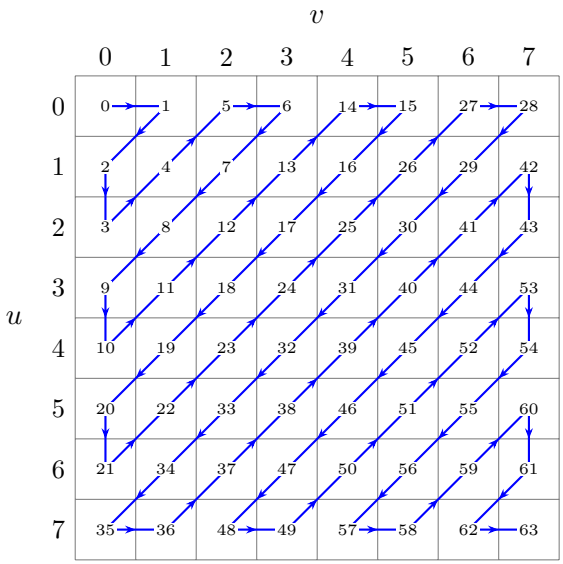


Figure 4: Zigzag scan order of an 8×8 block. The path starts at $(0, 0)$ and ends at $(7, 7)$.

Here:

- r: number of zeros before the value
- s: number of bits needed for the value's magnitude

2.7 Huffman Coding

A Huffman code assigns short bit strings to frequent symbols and long ones to rare symbols. It is applied to the DC difference sizes and the (r, s) pairs. The value bits follow each symbol unchanged. A negative value is stored as the bitwise complement of its magnitude, so 3 is `11` and -3 is `00`.

Huffman coding is lossless. The tables are either the standard ones in ITU-T T.81 [2] or are built per image and stored in the file. The bit stream, with headers holding the image size, Q and the Huffman tables, is the `.jpg` file [8].

2.8 Decoding

The decoder applies the steps in reverse order:

- Huffman decoding
- Inverse zigzag
- Multiplication by Q
- Inverse DCT
- Adding 128
- Chroma upsampling
- YCbCr to RGB conversion

The output approximates the original. Subsampling and quantization cannot be inverted.

2.9 Worked Example

This example follows the first MCU of the lighthouse image (512×768) at quality 50. The program writes it with `--trace trace.txt` (`trace.txt`). Only the first of the six blocks (the top-left Y block) is shown in full.

Color conversion of the pixel $(0, 0)$, with $(R, G, B) = (75, 93, 94)$:

$$\begin{aligned} Y &= 22.4 + 54.6 + 10.7 = 87.7 \\ C_b &= 128 - 12.7 - 30.8 + 47.0 = 131.5 \\ C_r &= 128 + 37.5 - 38.9 - 7.6 = 118.9 \end{aligned}$$

C_b and C_r are then averaged over 2×2 groups. An MCU covers 16×16 pixels and has 4 Y blocks, 1 C_b block and 1 C_r block.

Level-shifted samples of the first Y block, rounded to 1 decimal place:

$$f - 128 = \begin{bmatrix} -40.3 & -37.1 & -39.1 & -36.1 & -30.5 & -33.2 & -33.2 & -30.5 \\ -40.3 & -39.1 & -41.2 & -40.1 & -37.2 & -34.5 & -30.5 & -33.2 \\ -38.6 & -37.3 & -40.1 & -40.1 & -37.2 & -34.5 & -30.5 & -30.5 \\ -30.6 & -37.3 & -41.4 & -40.1 & -37.2 & -36.1 & -31.5 & -31.5 \\ -29.3 & -36.3 & -40.1 & -39.1 & -39.1 & -41.2 & -35.5 & -33.1 \\ -27.9 & -34.7 & -39.1 & -43.1 & -40.4 & -38.7 & -35.8 & -34.7 \\ -31.9 & -30.7 & -38.5 & -36.1 & -40.1 & -39.1 & -36.1 & -33.4 \\ -29.1 & -31.9 & -34.7 & -33.5 & -34.5 & -39.1 & -40.1 & -39.1 \end{bmatrix}$$

DCT:

$$F = \begin{bmatrix} -288.1 & -4.1 & 15.1 & 4.4 & 3.7 & -0.6 & -3.9 & 0.5 \\ -0.3 & -18.1 & -4.3 & 0.8 & -2.8 & -1.5 & -0.7 & 1.2 \\ 2.9 & 1.2 & -10.0 & -2.5 & -0.8 & -3.2 & -0.8 & -1.6 \\ -1.1 & 1.2 & 2.7 & 0.1 & 1.3 & -1.0 & 0.6 & 0.5 \\ 3.0 & 3.5 & -3.4 & 0.7 & 4.8 & -0.2 & 0.3 & 1.2 \\ 3.4 & -2.3 & 0.8 & -0.6 & -0.0 & -1.5 & -0.7 & -1.8 \\ 1.0 & 0.8 & -0.8 & 1.0 & -0.2 & -1.5 & 1.9 & 1.8 \\ 0.5 & -0.5 & -1.3 & -3.7 & 0.7 & -0.8 & -1.4 & -1.4 \end{bmatrix}$$

Quantization with the luminance Q above:

$$F_q = \begin{bmatrix} -18 & 0 & 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & -2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Only 4 of the 64 values are non-zero. Zigzag scan:

$$-18,\ 0,\ 0,\ 0,\ -2,\ 2,\ \underbrace{0, \ldots, 0}_{6},\ -1,\ \underbrace{0, \ldots, 0}_{51}$$

Symbols, taking the previous DC as 0:

- DC
  Difference -18, size 5.

- AC
  $(3,2)$: -2, $(0,2)$:2, $(6,1)$: -1.
- End
  EOB.

The 64 values reduce to 1 DC difference, 3 (r,s) pairs and 1 EOB. With the Huffman codes built for this image, the block becomes (code, then value bits):

```
11110|01101  11110101|01  011|10  111010|0  010
```

which is 35 bits instead of $64 \times 8 = 512$ bits for the original samples.

2.10 PSNR

PSNR measures the loss of a lossy codec with a single number. It compares each value of the original image with the same value of the decoded image. The mean squared error (MSE) is

$$\mathrm{MSE} = \frac{1}{3WH} \sum_{c=1}^{3} \sum_{x=1}^{H} \sum_{y=1}^{W} \big(p(x,y,c) - \hat{p}(x,y,c)\big)^2$$

and the PSNR is

$$\mathrm{PSNR} = 10 \log_{10} \frac{255^2}{\mathrm{MSE}} \ \mathrm{dB}$$

Here:

- W, H: image width and height in pixels
- $p, \hat{p}$: original and decoded value of channel c (R, G or B) at the pixel
- 255: the largest value of an 8-bit sample, the peak

The logarithm turns the error into a scale where each 10 dB is a tenfold change in MSE. For example, an MSE of 1 gives $10 \log_{10} 255^2 \approx 48.1$ dB, and an MSE of 100 gives 28.1 dB. Identical images have an MSE of 0 and an infinite PSNR. For 8-bit photos, values above about 40 dB are hard to tell from the original, and values from 30 to 40 dB are usually good. PSNR counts every error equally, so it does not match human perception exactly.

3 Python Implementation

The encoder and decoder use only the Python standard library, except that Pillow loads the input image. The code follows the steps in Section 2. Three classes hold the code: `Color` (RGB and YCbCr conversion), `Matrix` (transpose and matrix product, used by the DCT) and `Jpeg`. In `Jpeg`, `encode` and `decode` run the steps one after the other and each step is a method. The methods both directions need (DCT, quantization, zigzag, symbols, Huffman codes) are shared.

All files of this assignment (code, tests, test image, trace and this report) are in the GitHub repository, including `JR_230557T.py`.

The input is a BMP or PNG file, loaded with Pillow as 24 bpp RGB (a BMP stores 24 bpp pixels directly). The encoder writes a real baseline JPEG file: JFIF header, quantization tables, Huffman tables and the scan data, with 4:2:0 subsampling. Any JPEG viewer can open it. The decoder reads such a file back to measure the loss. Run it with:

`python3 JR_230557T.py image.bmp [quality]` to compress into `name-quality.jpg`. It prints the file size, the compression ratio and the PSNR of the decoded result. Adding `--trace trace.txt` also writes the steps and the data of the first MCU to that file, as a worked example.

```python
"""Baseline JPEG encoder and decoder in native Python.

Usage: python3 JR_230557T.py image.bmp|png [quality] [--trace trace.txt]  (writes image-quality.jpg)
With --trace, the steps and the data of the first MCU are written there as a worked example.
"""
import argparse
import contextlib
import heapq
import math
import os
import struct
import sys
from collections import Counter
from collections.abc import Iterator
from typing import TextIO

from PIL import Image

Plane = list[list[float]]  # rows of samples; a block is a small plane (8x8)
Block = Plane
IntBlock = list[list[int]]  # quantized coefficients or a quantization table
BlockGrid = list[list[Block]]  # grid[block row][block column]
RgbRows = list[list[tuple[int, int, int]]]  # rows of (r, g, b)
CodedBlock = tuple[int, str, list[tuple[int, str]]]  # (DC size, DC bits, [(AC symbol, value bits)])
HuffmanTable = tuple[list[int], list]  # (length_counts, symbols) as stored in a JPEG file


def clamp(value: float, low: int, high: int) -> int:
    """Round to an integer and keep it within low..high."""
    return max(low, min(high, round(value)))


class Color:
    """JFIF color conversion between RGB and YCbCr."""

    @staticmethod
    def rgb_to_ycbcr(r: float, g: float, b: float) -> tuple[float, float, float]:
        # Y is the brightness. Cb and Cr are the blue and red differences, centered on 128.
        # The factors are those of JFIF, section "Conversion to and from RGB":
        # https://www.w3.org/Graphics/JPEG/jfif3.pdf
        y = 0.299 * r + 0.587 * g + 0.114 * b
        cb = 128 - 0.168736 * r - 0.331264 * g + 0.5 * b
        cr = 128 + 0.5 * r - 0.418688 * g - 0.081312 * b
        return y, cb, cr

    @staticmethod
    def ycbcr_to_rgb(y: float, cb: float, cr: float) -> tuple[float, float, float]:
        # The inverse of rgb_to_ycbcr (same JFIF section):
        # https://www.w3.org/Graphics/JPEG/jfif3.pdf
        r = y + 1.402 * (cr - 128)
        g = y - 0.344136 * (cb - 128) - 0.714136 * (cr - 128)
        b = y + 1.772 * (cb - 128)
        return r, g, b


class Matrix:
    """A matrix as a list of rows. Supports transpose() and the @ operator (matrix product)."""

    def __init__(self, rows: list[list[float]]) -> None:
        self.rows = rows

    def transpose(self) -> "Matrix":
        """Swap rows and columns: entry [i][j] becomes entry [j][i]."""
        return Matrix([list(column) for column in zip(*self.rows)])

    def __matmul__(self, other: "Matrix") -> "Matrix":
        """Matrix product: entry [i][j] is row i of self times column j of other."""
        columns = other.transpose().rows
        result = []
        for row in self.rows:
            result_row = []
            for column in columns:
                result_row.append(sum(x * y for x, y in zip(row, column)))
            result.append(result_row)
        return Matrix(result)


class Jpeg:
    """Baseline JPEG (JFIF, 4:2:0) encoder and decoder

    encode() and decode() run their steps one after the other, each step is a
    method. The methods both directions need (DCT, quantization, zigzag,
    symbols, Huffman codes) are shared.
    """

    # Plane 0 (Y) uses LUMA, planes 1 and 2 (Cb, Cr) use CHROMA. They are
    # Table K.1 and Table K.2 in Annex K of ITU-T T.81:
    # https://www.w3.org/Graphics/JPEG/itu-t81.pdf
    LUMA = [
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
```

```
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 109, 103, 77],
        [24, 35, 55, 64, 81, 104, 113, 92],
        [49, 64, 78, 87, 103, 121, 120, 101],
        [72, 92, 95, 98, 112, 100, 103, 99],
    ]
    CHROMA = [
        [17, 18, 24, 47, 99, 99, 99, 99],
        [18, 21, 26, 66, 99, 99, 99, 99],
        [24, 26, 56, 99, 99, 99, 99, 99],
        [47, 66, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
    ]

    # The zigzag scan order of an 8x8 block. Cells on the same anti-diagonal have
    # the same row + column. The scan visits diagonal 0, 1, 2, ... and alternates
    # the direction on each one (even diagonals run from bottom-left to top-right).
    # See Figure A.6 of ITU-T T.81: https://www.w3.org/Graphics/JPEG/itu-t81.pdf
    ZIGZAG_ORDER = []
    for diagonal in range(15):
        cells = []
        for row in range(8):
            if 0 <= diagonal - row < 8:
                cells.append((row, diagonal - row))
        ZIGZAG_ORDER += cells[::-1] if diagonal % 2 == 0 else cells
    del diagonal, cells, row

    # An AC symbol is one number: 16 * (zeros before the value) + (size of the
    # value). It fits in one byte because both parts are at most 15.
    # The EOB and ZRL symbols are defined in Annex F (F.1.2.2) of ITU-T T.81:
    # https://www.w3.org/Graphics/JPEG/itu-t81.pdf
    END_OF_BLOCK = 0  # all remaining coefficients are zero
    ZRL = 240  # 16 zeros in a row (15 * 16 + 0), followed by more coefficients

    # Huffman codes are at most 16 bits long (Annex C of ITU-T T.81, and the
    # code length limit of Annex K.2): https://www.w3.org/Graphics/JPEG/itu-t81.pdf
    MAX_LENGTH = 16
    RESERVED = object()  # a fake symbol, see huffman_table

    TRACED_BLOCKS = 6  # a trace shows the blocks of the first MCU

    def __init__(self, quality: int = 50, trace: TextIO | None = None) -> None:
        """quality is from 1 to 100. The decoder replaces the tables with the ones in the file.

        trace is an open text file. If given, the steps and the data of the first
        MCU are written to it, as a worked example.
        """
        self.trace = trace
        self.traced = 0  # blocks traced so far in the current encode or decode
        luma, chroma = self.scale_table(self.LUMA, quality), self.scale_table(self.CHROMA, quality)
        self.quality = quality
        self.tables = [luma, chroma, chroma]  # quantization tables of Y, Cb and Cr

        # DCT matrix: T[u][x] = C(u) * cos((2x + 1) * u * pi / 16), with C(0) = sqrt(1/8)
        # and C(u) = sqrt(2/8) otherwise. The 2D DCT of a block f is T f T^t.
        rows = []
        for u in range(8):
            scale = math.sqrt(1 / 8) if u == 0 else math.sqrt(2 / 8)
            row = []
            for x in range(8):
                row.append(scale * math.cos((2 * x + 1) * u * math.pi / 16))
            rows.append(row)
        self.matrix = Matrix(rows)
        self.matrix_transposed = self.matrix.transpose()

        # The scan data while decoding: a string of '0' and '1' and the read position.
        self.bits = ""
        self.position = 0

    def log(self, text: str) -> None:
        """Write text to the trace file, if there is one."""
        if self.trace:
            print(text, file=self.trace)

    @staticmethod
    def show(block: list[list]) -> str:
        """An 8x8 block as text, one row per line."""
        return "\n".join(" ".join("%8.2f" % v if isinstance(v, float) else "%8d" % v for v in row) for row in block)

    # -------------------------------------------------------------------
    # Encoding: one method per step.
    # -------------------------------------------------------------------

    def encode(self, image: Image.Image) -> bytes:
        """Returns the contents of the JPEG file as bytes."""
        width, height = image.size
        padded_width, padded_height = self.padded_size(width, height)
        self.traced = 0
        self.log("== Encoding ==\nimage %dx%d, padded to %dx%d, quality %d"
                 % (width, height, padded_width, padded_height, self.quality))
```

```
        self.log("luma quantization table:\n" + self.show(self.tables[0])
                 + "\nchroma quantization table:\n" + self.show(self.tables[1]))

        planes = self.convert_to_ycbcr(image)  # 1. Color conversion
        planes = self.pad_planes(planes, padded_width, padded_height)  # 2. Padding
        planes = self.subsample_chroma(planes)  # 3. Chroma subsampling
        coded_blocks = self.code_blocks(planes, padded_width, padded_height)  # 4. DCT to run-length
        dc_table, ac_table, scan = self.huffman_encode(coded_blocks)  # 5. Huffman coding
        return self.write_file(width, height, dc_table, ac_table, scan)  # 6. The file

    def convert_to_ycbcr(self, image: Image.Image) -> list[Plane]:
        """Image to three planes Y, Cb, Cr."""
        y_plane, cb_plane, cr_plane = [], [], []
        pixels = image.load()  # much faster than getpixel
        for y in range(image.height):
            y_row, cb_row, cr_row = [], [], []
            for x in range(image.width):
                r, g, b = pixels[x, y]
                luma, cb, cr = Color.rgb_to_ycbcr(r, g, b)
                y_row.append(luma)
                cb_row.append(cb)
                cr_row.append(cr)
            y_plane.append(y_row)
            cb_plane.append(cb_row)
            cr_plane.append(cr_row)
        self.log("\n-- 1. Color conversion --\npixel (0, 0): RGB %s -> Y %.2f, Cb %.2f, Cr %.2f"
                 % (image.getpixel((0, 0)), y_plane[0][0], cb_plane[0][0], cr_plane[0][0]))
        return [y_plane, cb_plane, cr_plane]

    def pad_planes(self, planes: list[Plane], width: int, height: int) -> list[Plane]:
        """Grow every plane to the padded size."""
        self.log("\n-- 2. Padding --\n%dx%d -> %dx%d (last column and row repeated)"
                 % (len(planes[0][0]), len(planes[0]), width, height))
        return [self.pad(plane, width, height) for plane in planes]

    def subsample_chroma(self, planes: list[Plane]) -> list[Plane]:
        """Cb and Cr to half size in both directions (Y stays at full size)."""
        if self.trace:
            self.log("\n-- 3. Chroma subsampling --\nCb top-left 2x2 (averaged to one sample):\n"
                     + self.show([row[:2] for row in planes[1][:2]]))
        return [planes[0], self.subsample(planes[1]), self.subsample(planes[2])]

    def code_blocks(self, planes: list[Plane], padded_width: int, padded_height: int
                    ) -> list[CodedBlock]:
        """Every 8x8 block in MCU order: level shift, DCT, quantization, zigzag, run-length.

        Returns one (DC size, DC bits, [(AC symbol, value bits)]) per block.
        """
        self.log("\n-- 4. Blocks of the first MCU: Y, Y, Y, Y, Cb, Cr --")
        grids = [self.split_into_blocks(plane) for plane in planes]
        previous_dc = [0, 0, 0]  # each plane has its own DC predictor
        coded_blocks = []
        for plane, row, column in self.mcu_order(padded_width, padded_height):
            coded_block, previous_dc[plane] = self._code_block(grids[plane][row][column], plane, previous_dc[plane])
            coded_blocks.append(coded_block)
        return coded_blocks

    def huffman_encode(self, coded_blocks: list[CodedBlock]
                       ) -> tuple[HuffmanTable, HuffmanTable, bytes]:
        """The DC sizes and the AC symbols get separate codes.

        Returns the DC table, the AC table (each is (length_counts, symbols)) and the scan data.
        """
        dc_symbols, ac_symbols = [], []
        for dc_size, _, acs in coded_blocks:
            dc_symbols.append(dc_size)
            for symbol, _ in acs:
                ac_symbols.append(symbol)
        dc_table, ac_table = self.huffman_table(dc_symbols), self.huffman_table(ac_symbols)
        dc_codes, ac_codes = self.huffman_codes(*dc_table), self.huffman_codes(*ac_table)

        if self.trace:
            self.log("\n-- 5. Huffman coding --\nDC table (length_counts, symbols): %s\nDC codes: %s\n"
                     "AC table (length_counts, symbols): %s\nAC codes: %s"
                     % (dc_table, dc_codes, ac_table, ac_codes))
            for i, (dc_size, dc_bits, acs) in enumerate(coded_blocks[:self.TRACED_BLOCKS]):
                self.log("block %d bits (code|value bits): DC %s|%s  AC %s" % (
                    i, dc_codes[dc_size], dc_bits, " ".join(ac_codes[s] + "|" + b for s, b in acs)))

        chunks = []
        for dc_size, dc_bits, acs in coded_blocks:
            chunks.append(dc_codes[dc_size] + dc_bits)
            for symbol, value_bits in acs:
                chunks.append(ac_codes[symbol] + value_bits)
        return dc_table, ac_table, self.pack_bits(chunks)

    def write_file(self, width: int, height: int, dc_table: HuffmanTable,
                   ac_table: HuffmanTable, scan: bytes) -> bytes:
        """The file: markers with the tables, then the scan.

        Each segment is built by _segment: JFIF header, 2 quantization tables (DQT),
        frame header (SOF0), 2 Huffman tables (DHT), scan header (SOS), scan data.
        The marker codes (0xD8 SOI, 0xDB DQT, 0xC0 SOF0, 0xC4 DHT, 0xDA SOS, 0xD9 EOI)
```

```
        are listed in Table B.1 of ITU-T T.81: https://www.w3.org/Graphics/JPEG/itu-t81.pdf
        The JFIF header (0xE0, "JFIF\0", version 1.01) is described in: https://www.w3.org/Graphics/JPEG/jfif3.pdf
        """
        return (b"\xFF\xD8"  # SOI, start of image
                + self._segment(0xE0, b"JFIF\x00\x01\x01\x00" + struct.pack(">HH", 1, 1) + b"\x00\x00")
                + self._segment(0xDB, b"\x00" + bytes(self.zigzag_scan(self.tables[0])))  # tables are
                + self._segment(0xDB, b"\x01" + bytes(self.zigzag_scan(self.tables[1])))  # in zigzag order
                # 8 bits per sample, 3 components; each is (id, sampling factors, quantization table)
                + self._segment(0xC0, b"\x08" + struct.pack(">HH", height, width)
                                + bytes([3, 1, 0x22, 0, 2, 0x11, 1, 3, 0x11, 1]))
                + self._segment(0xC4, b"\x00" + bytes(dc_table[0]) + bytes(dc_table[1]))
                + self._segment(0xC4, b"\x10" + bytes(ac_table[0]) + bytes(ac_table[1]))
                # 3 components, all on DC table 0 and AC table 0; all 64 coefficients
                + self._segment(0xDA, bytes([3, 1, 0, 2, 0, 3, 0, 0, 63, 0]))
                + scan
                + b"\xFF\xD9")  # EOI, end of image

    def _code_block(self, block: Block, plane: int, previous_dc: int
                    ) -> tuple[CodedBlock, int]:
        """Returns ((DC size, DC bits, [(AC symbol, value bits)]), the DC value of this block)."""
        # Level shift, see T.81 section A.3.1: https://www.w3.org/Graphics/JPEG/itu-t81.pdf
        shifted = self.shift_levels(block, -128)
        coefficients = self.dct_forward(shifted)
        quantized = self.quantize(coefficients, plane)
        scan = self.zigzag_scan(quantized)

        # The first value is the DC coefficient. It is stored as the
        # difference from the DC of the previous block of the same plane.
        dc_difference = scan[0] - previous_dc
        dc_size, dc_bits = self.symbol_size(dc_difference), self.value_bits(dc_difference)
        pairs = self.rle(scan[1:])
        acs = []
        for symbol, value in pairs:
            acs.append((symbol, self.value_bits(value)))

        if self.traced < self.TRACED_BLOCKS and self.trace:
            self.log("\nblock %d (%s)\nsamples:\n%s\nlevel shifted (-128):\n%s\nDCT:\n%s\nquantized:\n%s\n"
                     "zigzag: %s\nDC: %d - previous %d = difference %d, size %d, bits '%s'\n"
                     "AC (symbol = 16 * zeros + size, value, bits): %s"
                     % (self.traced, ("Y", "Cb", "Cr")[plane], self.show(block), self.show(shifted),
                        self.show(coefficients), self.show(quantized), scan, scan[0], previous_dc,
                        dc_difference, dc_size, dc_bits,
                        [(s, v, b) for (s, v), (_, b) in zip(pairs, acs)]))
            self.traced += 1
        return (dc_size, dc_bits, acs), scan[0]

    @staticmethod
    def _segment(marker: int, payload: bytes) -> bytes:
        """A marker segment: 0xFF, the marker, a 2-byte length and the payload."""
        return bytes([0xFF, marker]) + struct.pack(">H", len(payload) + 2) + payload  # the length counts itself

    # -----------------------------------------------------------------------
    # Decoding: one method per step.
    # -----------------------------------------------------------------------

    def decode(self, file_bytes: bytes) -> RgbRows:
        """Returns the image as rows of (r, g, b).

        This reads the file the encoder writes. Progressive files, restart
        markers and other subsampling are not supported.
        """
        self.traced = 0
        self.log("\n== Decoding ==")
        width, height, lookups, scan_tables, scan_start = self.read_headers(file_bytes)  # 1. Markers
        self.bits, self.position = self._scan_bits(file_bytes, scan_start), 0
        self.log("image %dx%d, scan data of %d bits\nquantization tables read from the file:\n%s\n%s"
                 % (width, height, len(self.bits), self.show(self.tables[0]), self.show(self.tables[1])))
        grids = self.read_blocks(width, height, lookups, scan_tables)  # 2. Huffman to blocks
        planes = self.rebuild_planes(grids)  # 3. Blocks to planes
        return self.convert_to_rgb(planes, width, height)  # 4. Color conversion

    def read_headers(self, file_bytes: bytes) -> tuple[int, int, dict, list[tuple[int, int]], int]:
        """Read the segments up to the start of the scan. Each one is the marker
        byte, a length, and a payload that depends on the marker.

        Stores the quantization tables of the file in self.tables. Returns
        (width, height, Huffman lookups, scan tables, position of the scan data).
        """
        if file_bytes[0:2] != b"\xFF\xD8":
            raise ValueError("not a JPEG file")

        quantization_tables = {}  # table id -> 8x8 table
        lookups = {}  # (class, id) -> {bit string: symbol}
        position = 2
        while True:
            if file_bytes[position] != 0xFF:
                raise ValueError("expected a marker")
            marker = file_bytes[position + 1]
            length = struct.unpack_from(">H", file_bytes, position + 2)[0]
            payload = file_bytes[position + 4:position + 2 + length]
            position += 2 + length

            if marker == 0xDB:
```

```
                for offset in range(0, len(payload), 65):
                    if payload[offset] // 16 != 0:
                        raise ValueError("only 8-bit quantization tables are supported")
                    values = list(payload[offset + 1:offset + 65])
                    quantization_tables[payload[offset] % 16] = self.zigzag_unscan(values)
            elif marker == 0xC0:
                width, height, component_tables = self._read_frame(payload)
            elif marker == 0xC4:
                self._read_huffman_tables(payload, lookups)
            elif marker == 0xDA:
                # (DC table id, AC table id) of each component
                scan_tables = [divmod(payload[2 + 2 * i], 16) for i in range(payload[0])]
                break
            if marker in (0xDD, 0xC1, 0xC2, 0xC3, 0xC5, 0xC6, 0xC7, 0xC9, 0xCA, 0xCB):
                raise ValueError("only baseline JPEG without restart markers is supported")
            # Other markers (JFIF header, comments, Exif) are skipped.

        self.tables = [quantization_tables[table_id] for table_id in component_tables]
        return width, height, lookups, scan_tables, position

    def read_blocks(self, width: int, height: int, lookups: dict, scan_tables: list[tuple[int, int]]
                    ) -> list[BlockGrid]:
        """Decode the scan block by block, in MCU order. Returns the grid of 8x8 blocks of each plane."""
        padded_width, padded_height = self.padded_size(width, height)
        grids = []
        for size in (8, 16, 16):  # a Y block covers 8 pixels, a Cb or Cr block 16
            grids.append([[None] * (padded_width // size) for _ in range(padded_height // size)])
        previous_dc = [0, 0, 0]
        for plane, row, column in self.mcu_order(padded_width, padded_height):
            grids[plane][row][column], previous_dc[plane] = self._read_block(
                plane, previous_dc[plane], lookups, scan_tables)
        return grids

    def rebuild_planes(self, grids: list[BlockGrid]) -> list[Plane]:
        """Join the blocks into planes. Cb and Cr are back to full size by upsampling."""
        return [self.join_blocks(grids[0]),
                self.upsample(self.join_blocks(grids[1])),
                self.upsample(self.join_blocks(grids[2]))]

    def convert_to_rgb(self, planes: list[Plane], width: int, height: int
                       ) -> RgbRows:
        """Planes to rows of (r, g, b). Only the first width x height pixels are
        kept, which drops the padding."""
        rgb = []
        for y in range(height):
            row = []
            for x in range(width):
                r, g, b = Color.ycbcr_to_rgb(planes[0][y][x], planes[1][y][x], planes[2][y][x])
                row.append((clamp(r, 0, 255), clamp(g, 0, 255), clamp(b, 0, 255)))
            rgb.append(row)
        return rgb

    @staticmethod
    def _read_frame(payload: bytes) -> tuple[int, int, list[int]]:
        height, width = struct.unpack_from(">HH", payload, 1)
        if payload[0] != 8 or payload[5] != 3:
            raise ValueError("need 8-bit YCbCr")

        # Each component is 3 bytes: id, sampling factors, quantization table id.
        if [payload[7], payload[10], payload[13]] != [0x22, 0x11, 0x11]:
            raise ValueError("only 4:2:0 subsampling is supported")
        return width, height, [payload[8], payload[11], payload[14]]

    def _read_huffman_tables(self, payload: bytes, lookups: dict) -> None:
        offset = 0
        while offset < len(payload):
            table_class, table_id = divmod(payload[offset], 16)
            length_counts = list(payload[offset + 1:offset + 17])
            n_symbols = sum(length_counts)
            codes = self.huffman_codes(length_counts, list(payload[offset + 17:offset + 17 + n_symbols]))
            lookups[(table_class, table_id)] = {code: symbol for symbol, code in codes.items()}
            offset += 17 + n_symbols

    @staticmethod
    def _scan_bits(file_bytes: bytes, start: int) -> str:
        """The scan data as a string of '0' and '1'. The scan ends at the EOI
        marker. The stuffed 0x00 after each 0xFF is removed."""
        data = file_bytes[start:file_bytes.index(b"\xFF\xD9", start)].replace(b"\xFF\x00", b"\xFF")
        return "".join(format(byte, "08b") for byte in data)

    def _read_block(self, plane: int, previous_dc: int, lookups: dict, scan_tables: list[tuple[int, int]]
                    ) -> tuple[Block, int]:
        """Decode the next block of the scan. Returns (samples, its DC value)."""
        dc_id, ac_id = scan_tables[plane]
        ac_lookup = lookups[(1, ac_id)]
        dc_size = self.read_symbol(lookups[(0, dc_id)])
        scan = [previous_dc + self.bits_value(self.read_bits(dc_size))]  # the 64 values in zigzag order

        while len(scan) < 64:
            symbol = self.read_symbol(ac_lookup)
            if symbol == self.END_OF_BLOCK:
                break
            zeros, value_size = divmod(symbol, 16)  # ZRL is 15 zeros and a "value" 0, 16 zeros in all
```

```
            scan += [0] * zeros + [self.bits_value(self.read_bits(value_size))]
        scan += [0] * (64 - len(scan))  # after EOB, the rest of the block is zeros

        # Inverse zigzag, dequantize, inverse DCT, undo the level shift.
        coefficients = self.dequantize(self.zigzag_unscan(scan), plane)
        samples = self.shift_levels(self.dct_inverse(coefficients), 128)
        if self.traced < self.TRACED_BLOCKS and self.trace:
            self.log("\nblock %d (%s)\nzigzag (DC is the sum with the previous DC): %s\ndequantized:\n%s\n"
                     "samples after inverse DCT and +128:\n%s"
                     % (self.traced, ("Y", "Cb", "Cr")[plane], scan, self.show(coefficients), self.show(samples)))
            self.traced += 1
        return samples, scan[0]

    @staticmethod
    def padded_size(width: int, height: int) -> tuple[int, int]:
        """Size rounded up to a multiple of 16 (4:2:0 needs 16x16 groups)."""
        return (width + 15) // 16 * 16, (height + 15) // 16 * 16

    @staticmethod
    def pad(plane: Plane, width: int, height: int) -> Plane:
        """Grow a plane (a list of rows of samples) to width x height by repeating its last column and row."""
        padded = []
        for row in plane:  # extend every row to the right
            padded.append(list(row) + [row[-1]] * (width - len(row)))
        while len(padded) < height:  # copies of the last row at the bottom
            padded.append(list(padded[-1]))
        return padded

    @staticmethod
    def subsample(plane: Plane) -> Plane:
        """4:2:0 subsampling: replace each 2x2 group by its average."""
        result = []
        for y in range(0, len(plane), 2):  # (y, x) is the top-left sample of each group
            row = []
            for x in range(0, len(plane[0]), 2):
                average = 0.25 * (plane[y][x] + plane[y][x + 1] + plane[y + 1][x] + plane[y + 1][x + 1])
                row.append(average)
            result.append(row)
        return result

    @staticmethod
    def upsample(plane: Plane) -> Plane:
        """Repeat every sample twice horizontally and twice vertically."""
        result = []
        for row in plane:
            wide = []
            for value in row:
                wide += [value, value]
            result += [wide, list(wide)]
        return result

    @staticmethod
    def split_into_blocks(plane: Plane) -> BlockGrid:
        """Cut a plane into 8x8 blocks. Returns a grid: grid[block row][block column]."""
        grid = []
        for top in range(0, len(plane), 8):
            grid_row = []
            for left in range(0, len(plane[0]), 8):
                grid_row.append([plane[y][left:left + 8] for y in range(top, top + 8)])
            grid.append(grid_row)
        return grid

    @staticmethod
    def join_blocks(grid: BlockGrid) -> Plane:
        """Put a grid of 8x8 blocks back into a plane."""
        plane = []
        for grid_row in grid:
            for y in range(8):
                row = []
                for block in grid_row:
                    row += block[y]
                plane.append(row)
        return plane

    @classmethod
    def mcu_order(cls, padded_width: int, padded_height: int) -> Iterator[tuple[int, int, int]]:
        """The blocks of the scan as (plane, block row, block column), in MCU order.

        A block is 8x8 samples of one plane. The tuple says which block it is:
          plane        0 = Y (brightness), 1 = Cb, 2 = Cr
          block row    row of the block in the block grid of that plane
          block column column of the block in the block grid of that plane
        The grids differ in size: Cb and Cr have half as many block rows and
        columns as Y. The encoder reads grids[plane][row][column] for each tuple.

        An MCU (minimum coded unit) covers a 16x16 patch of the image, which
        is 256 pixels:
          Y  has one sample per pixel: 256 samples = 4 blocks (2 across, 2 down).
          Cb has one sample per 2x2 pixels: 64 samples = 1 block for the patch.
          Cr has one sample per 2x2 pixels: 64 samples = 1 block for the patch.
        So all four Y blocks share the one Cb block and the one Cr block.

            16 pixels
```

```
          +-------+-------+
          | Y     | Y     |      The MCU stores 6 blocks in this order:
          | (0,0) | (0,1) |        Y top-left, Y top-right,
          +-------+-------+        Y bottom-left, Y bottom-right,
          | Y     | Y     |        Cb, Cr
          | (1,0) | (1,1) |      (Y positions are relative to the patch.)
          +-------+-------+

        The patches go left to right, then top to bottom.

        For a 32x16 image (two patches side by side) the first MCU gives
        (0, 0, 0) (0, 0, 1) (0, 1, 0) (0, 1, 1) (1, 0, 0) (2, 0, 0)
        and the second gives
        (0, 0, 2) (0, 0, 3) (0, 1, 2) (0, 1, 3) (1, 0, 1) (2, 0, 1).
        """
        for patch_row in range(padded_height // 16):
            for patch_column in range(padded_width // 16):
                # Y blocks are 8x8 pixels, so a patch has two block rows and two block columns.
                y_row, y_column = 2 * patch_row, 2 * patch_column
                yield 0, y_row, y_column
                yield 0, y_row, y_column + 1
                yield 0, y_row + 1, y_column
                yield 0, y_row + 1, y_column + 1
                # Cb and Cr blocks cover the whole patch, so one block each.
                yield 1, patch_row, patch_column
                yield 2, patch_row, patch_column

    @staticmethod
    def shift_levels(block: Block, offset: int) -> Block:
        """Add offset to every sample (-128 before the DCT, +128 after the inverse DCT)."""
        shifted = []
        for row in block:
            shifted.append([sample + offset for sample in row])
        return shifted

    def dct_forward(self, block: Block) -> Block:
        """Level-shifted samples to DCT coefficients."""
        return (self.matrix @ Matrix(block) @ self.matrix_transposed).rows

    def dct_inverse(self, coefficients: Block) -> Block:
        """DCT coefficients back to level-shifted samples."""
        return (self.matrix_transposed @ Matrix(coefficients) @ self.matrix).rows

    @staticmethod
    def scale_table(table: IntBlock, quality: int) -> IntBlock:
        """Scale a quantization table for a quality from 1 to 100, with the formula
        of the Independent JPEG Group (jpeg_quality_scaling in
        https://github.com/libjpeg-turbo/libjpeg-turbo/blob/main/jcparam.c).
        At quality 50 the table is unchanged."""
        scale = 5000 / quality if quality < 50 else 200 - 2 * quality
        scaled = []
        for row in table:
            scaled_row = []
            for divisor in row:
                value = int((divisor * scale + 50) // 100)
                scaled_row.append(min(255, max(1, value)))  # keep it within 1..255
            scaled.append(scaled_row)
        return scaled

    def quantize(self, coefficients: Block, plane: int) -> IntBlock:
        """Divide each DCT coefficient by its table entry and round."""
        # The only lossy step of the encoder: rounding throws information away.
        result = []
        for row, table_row in zip(coefficients, self.tables[plane]):
            # Halves round away from zero, like libjpeg (round() would pick the even number).
            result.append([int(math.copysign(math.floor(abs(c) / t + 0.5), c)) for c, t in zip(row, table_row)])
        return result

    def dequantize(self, quantized: IntBlock, plane: int) -> Block:
        """Multiply each value by its table entry (the decoder's step)."""
        result = []
        for row, table_row in zip(quantized, self.tables[plane]):
            result.append([q * t for q, t in zip(row, table_row)])
        return result

    @classmethod
    def zigzag_scan(cls, block: list[list]) -> list:
        """8x8 block to a list of 64 values."""
        return [block[row][column] for row, column in cls.ZIGZAG_ORDER]

    @classmethod
    def zigzag_unscan(cls, values: list) -> list[list]:
        """List of 64 values back to an 8x8 block."""
        block = [[0] * 8 for _ in range(8)]
        for (row, column), value in zip(cls.ZIGZAG_ORDER, values):
            block[row][column] = value
        return block

    @staticmethod
    def symbol_size(value: int) -> int:
        """Number of bits needed for the magnitude of value (0 for 0)."""
        return abs(value).bit_length()
```

```
    @classmethod
    def value_bits(cls, value: int) -> str:
        """Bits stored after a symbol. A negative value is stored as the
        complement of its magnitude, so 3 is '11' and -3 is '00'."""
        n_bits = cls.symbol_size(value)
        if n_bits == 0:
            return ""
        return format(value + (value < 0) * (2 ** n_bits - 1), "0%db" % n_bits)

    @staticmethod
    def bits_value(bits: str) -> int:
        """Inverse of value_bits."""
        if bits == "":
            return 0
        value = int(bits, 2)
        return value if bits[0] == "1" else value - 2 ** len(bits) + 1  # a leading 0 marks a negative value

    @classmethod
    def rle(cls, ac_values: list[int]) -> list[tuple[int, int]]:
        """Run-length code the 63 AC values of a block.

        Returns a list of (symbol, value). Runs longer than 15 emit ZRL.
        A tail of zeros becomes EOB.
        """
        symbols = []
        zeros = 0  # zeros seen since the last non-zero value
        for value in ac_values:
            if value == 0:
                zeros += 1
                continue
            while zeros > 15:  # a symbol can only hold a run of up to 15
                symbols.append((cls.ZRL, 0))
                zeros -= 16
            symbols.append((zeros * 16 + cls.symbol_size(value), value))  # run and size share one symbol
            zeros = 0
        if zeros > 0:
            symbols.append((cls.END_OF_BLOCK, 0))
        return symbols

    @classmethod
    def huffman_table(cls, symbols: list) -> HuffmanTable:
        """Build the Huffman code of a list of symbols. Frequent symbols get shorter codes.

        A code is stored in the form a JPEG file uses: (length_counts, symbols).
        length_counts[i] is the number of codes that are i + 1 bits long and
        symbols lists the symbols in order of code length. The codes follow
        from these two lists alone (see huffman_codes).
        """
        counts = Counter(symbols)  # how often each symbol occurs
        if not counts:
            return [0] * cls.MAX_LENGTH, []

        lengths = cls._find_lengths(counts)

        # bits[n] is the number of symbols with a code of n bits.
        bits = [0] * (max(lengths.values()) + 1)
        for length in lengths.values():
            bits[length] += 1

        # The reserved symbol has the longest code. Its code would be all 1
        # bits, which JPEG does not allow, so it is dropped again at the end.
        bits = cls._limit_lengths(bits)
        bits[max(i for i, n in enumerate(bits) if n)] -= 1
        bits += [0] * (cls.MAX_LENGTH + 1 - len(bits))  # an entry for every length

        # The symbols go in order of their original code length, then by value.
        return bits[1:cls.MAX_LENGTH + 1], sorted(counts, key=lambda s: (lengths[s], s))

    @staticmethod
    def huffman_codes(length_counts: list[int], symbols: list) -> dict:
        """The bit string of each symbol (canonical Huffman code)."""
        # Codes of one length are consecutive numbers. The first code of the
        # next length is the next number, doubled.
        codes = {}
        code = index = 0
        for length, count in enumerate(length_counts, 1):
            for _ in range(count):
                codes[symbols[index]] = format(code, "0%db" % length)
                code += 1
                index += 1
            code *= 2
        return codes

    @classmethod
    def _find_lengths(cls, counts: Counter) -> dict:
        """Huffman's algorithm. Returns {symbol: code length}.

        The reserved symbol is added with count 0, so it gets the longest code.
        """
        lengths = dict.fromkeys(counts, 0)
        lengths[cls.RESERVED] = 0
        # Each heap entry is (count, tie breaker, symbols in this group). The
        # tie breaker stops Python from comparing the lists.
```

```
        heap = [(counts[symbol], tie, [symbol]) for tie, symbol in enumerate(sorted(counts))]
        heap.append((0, len(heap), [cls.RESERVED]))
        heapq.heapify(heap)

        tie = len(heap)
        while len(heap) > 1:
            count_a, _, group_a = heapq.heappop(heap)  # the two rarest groups
            count_b, _, group_b = heapq.heappop(heap)
            for symbol in group_a + group_b:
                lengths[symbol] += 1  # every symbol in the groups moves one level deeper
            heapq.heappush(heap, (count_a + count_b, tie, group_a + group_b))
            tie += 1
        return lengths

    @classmethod
    def _limit_lengths(cls, bits: list[int]) -> list[int]:
        """Shorten codes longer than 16 bits (procedure of ITU-T T.81, Annex K.2, Figure K.3:
        https://www.w3.org/Graphics/JPEG/itu-t81.pdf).

        Two codes of the longest length are replaced by one code one level
        up, and one code of a shorter length is split into two longer ones.
        """
        longest = len(bits) - 1
        while longest > cls.MAX_LENGTH:
            if bits[longest] == 0:
                longest -= 1
                continue
            shorter = longest - 2
            while bits[shorter] == 0:
                shorter -= 1
            bits[longest] -= 2
            bits[longest - 1] += 1
            bits[shorter + 1] += 2
            bits[shorter] -= 1
        return bits

    @staticmethod
    def pack_bits(chunks: list[str]) -> bytes:
        """Join strings of '0' and '1' into the bytes JPEG scan data needs.
        The last byte is filled with 1 bits."""
        bits = "".join(chunks)
        bits += "1" * (-len(bits) % 8)
        data = bytearray()
        for i in range(0, len(bits), 8):
            data.append(int(bits[i:i + 8], 2))
            if data[-1] == 255:
                data.append(0)  # 0xFF starts a marker, so a data byte 0xFF is followed by 0x00
                # (byte stuffing, T.81 section B.1.1.5: https://www.w3.org/Graphics/JPEG/itu-t81.pdf)
        return bytes(data)

    def read_bits(self, count: int) -> str:
        """The next count bits of self.bits."""
        self.position += count
        return self.bits[self.position - count:self.position]

    def read_symbol(self, lookup: dict[str, int]) -> int:
        """Read bits one at a time until they match a Huffman code.

        lookup maps a bit string to its symbol.
        """
        code = ""
        while code not in lookup:
            code += self.read_bits(1)
        return lookup[code]

    @staticmethod
    def psnr(original: Image.Image, decoded: RgbRows) -> float:
        """Peak signal-to-noise ratio in dB between the original image and the decoded rows."""
        # Sum of the squared differences over all pixels and all three colors.
        squared_error = 0
        pixels = original.load()
        for y, decoded_row in enumerate(decoded):
            for x, decoded_pixel in enumerate(decoded_row):
                for a, b in zip(pixels[x, y], decoded_pixel):
                    squared_error += (a - b) ** 2
        if squared_error == 0:
            return float("inf")
        # PSNR = 10 * log10(255^2 / MSE), where MSE = squared_error / n_values.
        n_values = original.width * original.height * 3
        return 10 * math.log10(255 * 255 * n_values / squared_error)


if __name__ == "__main__":
    # Encode the image and write it next to the input as name-quality.jpg.
    parser = argparse.ArgumentParser(description="Baseline JPEG encoder and decoder.")
    parser.add_argument("source", help="input image (bmp or png)")
    parser.add_argument("quality", nargs="?", type=int, default=50, help="1 to 100 (default 50)")
    parser.add_argument("--trace", metavar="FILE", help="write a worked example of the first MCU to FILE")
    args = parser.parse_args()
    if not 1 <= args.quality <= 100:
        parser.error("quality (%d) must be in [1, 100]" % args.quality)

    image = Image.open(args.source).convert("RGB")
```

```
    with open(args.trace, "w") if args.trace else contextlib.nullcontext() as trace:
        jpeg = Jpeg(args.quality, trace)
        file_bytes = jpeg.encode(image)
        output_file = "%s-%d.jpg" % (os.path.splitext(args.source)[0], args.quality)
        with open(output_file, "wb") as f:
            f.write(file_bytes)

        # Decode the file again to measure the loss.
        ratio = image.width * image.height * 24 / (len(file_bytes) * 8)
        decoded = jpeg.decode(file_bytes)
    print("quality %d file size %d bytes, compression ratio %.2f, PSNR %.2f dB"
          % (args.quality, len(file_bytes), ratio, Jpeg.psnr(image, decoded)))
```

3.1 Results

The test image is a 512×768 photo of a lighthouse, which is 9 437 184 bits at 24 bpp. The BMP file is 1 179 702 bytes. Figure 5 shows the original and two compressed versions. Table 1 shows the size of the JPEG file and the quality of the decoded image at ten quality settings.

A lower quality gives a larger divisor table, so the file shrinks and the PSNR falls. The ratio is the size of the pixel data divided by the size of the JPEG file. Quality 50 already gives a ratio of about 29, and quality 90 a ratio of about 10. Quality 100 gives the largest file and a ratio of only 3.8, because the divisors are all 1 and only rounding loss remains. Decoding the files with the image tools of the operating system gives the same PSNR, which confirms that they are valid JPEG files.



Figure 5: The test image: original (left), quality 50 (middle, 28.6:1) and quality 10 (right, 100.1:1).

3.2 Tests

The tests are in `test_JR_230557T.py`. The file contents are not included in this report to keep this short. The tests check for color conversion, the DCT, the quantization, the zigzag order and the run-length symbols, plane operations, the bit coding, the Huffman code and its 16-bit limit, the byte stuffing, the image loader, the PSNR, JPEG file's markers and the stored tables. Encode–decode round trips run on the lighthouse photo and on a small crop of it whose size is not a multiple of 16. Run them with `python3 -m unittest`. The real-image tests need `lighthouse.bmp` in the same folder.

Quality	File size (bytes)	Compression ratio	PSNR (dB)
10	11 788	100.1	26.8
20	20 997	56.2	29.3
30	28 814	40.9	30.6
40	35 261	33.5	31.6
50	41 297	28.6	32.3
60	47 826	24.7	33.0
70	57 674	20.4	34.0
80	74 276	15.9	35.4
90	112 754	10.5	38.1
100	312 866	3.8	46.1

Table 1: Compression of the lighthouse image.

4 Conclusion

The encoder and decoder were written in native Python and produce valid baseline JPEG files that other viewers open. Chroma subsampling and quantization are the only lossy steps, and quantization is the one the quality setting controls. On the lighthouse image, quality 50 gives a ratio of 28.6 at 32.3 dB, quality 90 a ratio of 10.5 at 38.1 dB, and quality 10 a ratio of 100.1 at 26.8 dB. The quality setting therefore trades file size against PSNR, and moderate settings keep the image close to the original at a fraction of its size.

References

[1] G. K. Wallace, "The JPEG still picture compression standard," *IEEE Trans. Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992. https://doi.org/10.1109/30.125072

[2] ITU-T Recommendation T.81 | ISO/IEC 10918-1, *Digital compression and coding of continuous-tone still images: Requirements and guidelines*, 1992. https://www.w3.org/Graphics/JPEG/itu-t81.pdf

[3] E. Hamilton, "JPEG File Interchange Format, Version 1.02," C-Cube Microsystems, Sept. 1992. https://www.w3.org/Graphics/JPEG/jfif3.pdf

[4] N. Ahmed, T. Natarajan and K. R. Rao, "Discrete cosine transform," *IEEE Trans. Computers*, vol. C-23, no. 1, pp. 90–93, Jan. 1974. https://doi.org/10.1109/T-C.1974.223784

[5] CIPA, *Exchangeable image file format for digital still cameras: Exif Version 2.32*, CIPA DC-008, 2019. https://www.cipa.jp/std/documents/e/DC-X008-Translation-2019-E.pdf

[6] ITU-T Recommendation T.84 | ISO/IEC 10918-3, *Digital compression and coding of continuous-tone still images: Extensions*, 1996. https://www.itu.int/rec/T-REC-T.84

[7] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952. https://doi.org/10.1109/JRPROC.1952.273898

[8] W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. New York: Van Nostrand Reinhold, 1992. https://openlibrary.org/isbn/0442012721

[9] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018. https://openlibrary.org/isbn/9780133356724